GB-PIFNO: Execution Plan

Dynamic Gradient-Balanced Physics-Informed Fourier Neural Operator for 2D Turbulent Navier–Stokes

Status as of: August 2026 **Environment:** Google Colab (PyTorch, T4/A100 GPU)

---

1. Project Summary

Problem. Standard Fourier Neural Operators (FNO) overfit and over-smooth high-frequency turbulent structures when training data is sparse (e.g. 5% sensor coverage) or noisy. Standard Physics-Informed FNO (Static PIFNO) tries to fix this by adding a PDE-residual loss term, but suffers from **gradient stiffness**: the PDE residual gradient magnitude overwhelms the data gradient magnitude during backpropagation, often collapsing the model to a trivial near-zero output.

Proposed solution — GB-PIFNO. At every training step, compute the exact L_2 norm of the gradients of both loss terms with respect to network weights, and dynamically reweight the PDE loss so the two gradient magnitudes stay balanced:

$$G_{data} = \|\nabla_\theta \mathcal{L}_{data}\|_2, \qquad G_{pde} = \|\nabla_\theta \mathcal{L}_{pde}\|_2$$

$$\hat{\lambda}_{pde}^{(t)} = \frac{G_{data}}{G_{pde}}, \qquad \lambda_{pde}^{(t+1)} = (1-\alpha)\,\lambda_{pde}^{(t)} + \alpha\,\hat{\lambda}_{pde}^{(t)}$$

A third loss term, a **Sobolev H^1 spectral penalty**, is also included to explicitly penalize blurred/incorrect spatial gradients (not just wrong values), computed in Fourier space:

$$\mathcal{F}\left(\frac{\partial u}{\partial x}\right) = ik_x \cdot \mathcal{F}(u)$$

Governing equation (2D incompressible Navier–Stokes, vorticity form):

$$\frac{\partial \omega}{\partial t} + u \cdot \nabla \omega = \nu \nabla^2 \omega + f$$

Central claim to prove: GB-PIFNO preserves the turbulent energy cascade (Kolmogorov $E(k) \propto k^{-3}$) and enstrophy dissipation under sparse/noisy observations, where data-only models (U-Net, FNO) and Static PIFNO all fail.

---

2. Models In Scope

Tier	Model	Role	Status
1	2D U-Net	Data-only spatial baseline	Architecture done
1	Standard FNO	Data-only spectral baseline	Architecture done
1	Static PIFNO ($\lambda = 1$)	Fixed-weight physics baseline	**Not started**
1	**GB-PIFNO** (ours)	Dynamic gradient-balanced physics model	**Not started**
2	DeepONet	Branch/trunk operator, architecturally distinct	Not started
2	U-NO *or* F-FNO (pick one)	FNO variant, secondary comparison	Not started
3	PINO, PINTO, CNO	Cite from literature; do not re-implement/re-train (compute + scope reasons — see §6)	Cite only

Note on PINO: architecturally, PINO (Li et al. 2021) *is* an FNO trained with a fixed-weight PDE residual loss — i.e., it is equivalent to our Static PIFNO baseline. We cite PINO as the origin of that idea and present Static PIFNO as its reproduction in our setting, rather than implementing it twice.

---

3. Dataset

- **Source:** Original FNO benchmark (Li et al., NeurIPS 2020), not PDEBench — chosen deliberately over PDEBench's 512×512 `NS_Incom` set (2.3 TB total) due to Colab VRAM constraints, since GB-PIFNO's dynamic balancing requires double-backpropagation (`torch.autograd.grad(..., create_graph=True)`), which roughly doubles activation memory versus standard training.
- **File:** `NavierStokes_V1e-5_N1200_T20.mat` — ν = 1e-5 (**Re ≈ 100,000**, fully turbulent regime), chosen over the easier ν=1e-3 set specifically because the paper's central claim (preserving the turbulent cascade) needs genuinely turbulent ground truth to be a meaningful test.
- **Format:** MATLAB v7.3-compatible / legacy `.mat`, key `'u'`, shape `(1200, 64, 64, 20)` — 1200 trajectories, 64×64 grid, 20 timesteps. (Keys `'a'` and `'t'` also present; `'a'` is redundant with `u[...,0]` and can be ignored.)
- **Split convention** (matches original FNO repo): first 1000 trajectories train, last 200 test. Input = first 10 timesteps, target = next 10 timesteps ($T_{in} = T_{out} = 10$).
- **Forcing term** — confirmed fixed, time-independent, baked into the data generation:

$$f(x, y) = 0.1\,(\sin(2\pi(x+y)) + \cos(2\pi(x+y)))$$

This **must** be included in the PDE residual computation (Phase 4) or the physics loss will be systematically biased.

- **Verified working load path:** Drive-mount → unzip inside Colab's local `/content/` disk (not inside Drive itself, for I/O speed) → `scipy.io.loadmat`. Confirmed working; do not re-attempt local Windows unzip + browser re-upload (caused truncation in earlier attempts).

---

4. What Has Been Completed So Far

4.1 Data pipeline (done, verified)

- Google Drive mount → Colab-local unzip → `scipy.io.loadmat` load path confirmed working.
- Verified: `u.shape == (1200, 64, 64, 20)`, dtype float32, value range ≈ [-3.76, 3.77], visually confirmed turbulent (fine-grained eddies, not smooth blobs) at later timesteps.
- `NSVorticityDataset` (PyTorch `Dataset`) + `DataLoader`, train/test split 1000/200, $T_{in} = T_{out} = 10$.

4.2 Model architectures (done, smoke-tested)

- **`SpectralConv2d`** + **`FNO2d`** (`fno.py`): modes=12, width=32, 4 Fourier layers, ~1.19M params. This exact class is reused unchanged for FNO, Static PIFNO, and GB-PIFNO — only the loss function differs between them, isolating the effect of the physics loss from architecture changes.
- **`UNet2d`** (`unet.py`): 4-level encoder/decoder, base width 32, ~7.77M params. Purely spatial-convolutional baseline — architecturally distinct inductive bias (local receptive field growth vs. FNO's global spectral mixing) for a meaningful baseline contrast.

4.3 Evaluation harness (done, smoke-tested)

`eval_metrics.py` — shared, model-agnostic, used identically for all models:

- `relative_l2_error` — standard spatial relative L_2
- `fourier_gradients` — exact spatial derivatives via FFT ($\mathcal{F}(\partial u/\partial x) = ik_x\hat{u}$); **this exact function will be reused in Phase 4 for the PDE residual**, keeping eval and physics-loss derivative computation consistent
- `h1_relative_error` — Sobolev H^1 error; catches "same L_2, but blurred" failure modes that plain L_2 misses
- `radial_energy_spectrum` — 1D angle-averaged $E(k)$; the central diagnostic for the Kolmogorov k^{-3} claim
- `enstrophy` — $\Omega(t) = \frac{1}{2}\int \omega^2\, dx$, per-sample-per-timestep

Not yet implemented: PDE residual norm $\|\mathcal{R}_{pde}\|$ — deferred to Phase 4 since it requires design decisions about time-derivative handling (finite-difference vs. autoregressive) tied to the training loop.

4.4 Sparsity stress-test infrastructure (done, smoke-tested)

`sparsity.py` — implements the sensor-sparsity ablation described in the proposal:

- Random **pixel-level** mask (not timestep dropout), same observed locations held fixed across all T_{in} input frames (models a fixed sensor network, not teleporting sensors)
- Zero-fill unobserved pixels + **explicit binary mask channel** concatenated to input (so the model can distinguish "observed zero" from "missing"), which changes effective input channels from T_{in} to $T_{in} + 1$

4.5 Data-only training loop (done, smoke-tested)

`train.py` — `train_data_only()`, shared between U-Net and FNO (identical I/O convention once sparsity masking is applied). Logs train MSE loss, test relative L_2, test relative H^1 per epoch. Adam + StepLR schedule.

4.6 Notebook coherence fixes (done)

Corrected cell-ordering bugs from copy-pasting standalone `.py` files into one notebook: removed inter-cell `import` statements (unnecessary — all pasted code shares one global namespace in a notebook), removed dead `__main__` smoke-test blocks, fixed `device` being referenced before definition, fixed the training-call cell being placed before `train_data_only` was defined.

→ At this point, the team can run real Tier-1 data-only baselines (U-Net, FNO) at any sparsity level and get real relative-L_2/H^1 numbers.

---

5. Remaining Work — Phased Plan

Phase 4 — PDE Residual + Static PIFNO

Owner suggestion: whoever is most comfortable with the Fourier-derivative math.

1. Extend `fourier_gradients` (already exists) to also compute the Laplacian $\nabla^2\omega$ (second derivative in Fourier space: multiply by $-(k_x^2 + k_y^2)$).
2. Compute the **time derivative** $\partial\omega/\partial t$ — decide via finite-difference across the T_{out} output frames (simplest) vs. autoregressive single-step rollout (more physically faithful but heavier compute). **Recommend finite-difference first** for a working baseline, revisit if residual quality is poor.
3. Implement `advection_term = u·∇ω`, requiring the velocity field $u = (u_x, u_y)$ recovered from vorticity via the stream function ($\omega = -\nabla^2\psi, u = (\partial\psi/\partial y, -\partial\psi/\partial x)$) — this is the standard vorticity-streamfunction formulation, all computable in Fourier space.
4. Hard-code the **exact forcing term** $f(x, y) = 0.1(\sin(2\pi(x + y)) + \cos(2\pi(x + y)))$ from §3 — this is not optional, the residual is meaningless without it.

5. Assemble PDE residual: $\mathcal{R} = \partial\omega/\partial t + u \cdot \nabla\omega - \nu\nabla^2\omega - f$, loss $\mathcal{L}_{pde} = \|\mathcal{R}\|_2^2$.
6. New training loop (`train_static_pifno()`): total loss $= \mathcal{L}_{data} + \lambda\,\mathcal{L}_{pde}$ with fixed $\lambda = 1.0$. Expect this to demonstrate the gradient-stiffness failure mode described in the proposal (may collapse toward trivial output) — **this failure is itself an expected and useful result**, not a bug to "fix" — it's the paper's motivating evidence.
7. Add (`pde_residual_norm`) to (`eval_metrics.py`) now that the residual function exists.

Deliverable: Static PIFNO trained + evaluated across the sparsity grid, plus the gradient-stiffness failure documented (numerically and ideally visually).

Phase 5 — GB-PIFNO (the core contribution)

Owner suggestion: whoever owns Phase 4 should also lead this — same codebase.

1. Implement per-step gradient norm computation: (`torch.autograd.grad(L_data, model.parameters(), retain_graph=True, create_graph=False)`) and similarly for (`L_pde`), then take the L_2 norm across all parameter gradients.
    - **VRAM note:** use (`create_graph=False`) unless you need to backprop *through* the balancing step itself — likely unnecessary here since λ update is just an EMA, not a learned parameter. This significantly reduces memory vs. full double-backprop.
2. Implement the EMA update: $\lambda^{(t+1)} = (1-\alpha)\lambda^{(t)} + \alpha \cdot (G_{data}/G_{pde})$. Expose α as a hyperparameter (start with $\alpha = 0.1$, tune later).
3. Also wire in the Sobolev H^1 loss term from the original proposal (separate from the PDE residual — confirm with the team whether H^1 gets its own weight or is folded into $\mathcal{L}_{data}$).
4. New training loop (`train_gb_pifno()`), logging $\lambda_{pde}^{(t)}, G_{data}, G_{pde}$ every step/epoch — **this log is exactly what produces the "Gradient Norm Pathologies" diagnostic plot** (Static PIFNO vs. GB-PIFNO gradient norms over epochs) called out as a key figure in the proposal.
5. Train across the same sparsity/noise grid as the other three models for direct comparison.

Deliverable: GB-PIFNO trained + evaluated, plus the gradient-norm-over-time diagnostic plot proving the mechanism works.

Phase 6 — DeepONet baseline

Owner suggestion: can be parallelized with Phase 4/5 by a second team member, since it's architecturally independent.

1. Implement branch net (encodes the sparse/masked input field) + trunk net (encodes query coordinates), standard DeepONet formulation.
2. Reuse the same (`sparsity.py`) masking and (`eval_metrics.py`) harness — no changes needed there.

3. Train data-only (DeepONet is not being physics-informed in this project's scope) across the sparsity grid.

Deliverable: DeepONet baseline numbers, added to the comparison table.

Phase 7 — Second spectral variant: U-NO or F-FNO (pick one)

Team decision needed: these are both "FNO + a modification" (U-NO adds multi-scale U-shaped structure; F-FNO factorizes the spectral convolution for memory efficiency). Recommend picking **one**, not both, since they'd otherwise dilute review attention without adding much narrative distinctiveness. Suggest deciding based on whichever has clearer reference code available.

Deliverable: one additional spectral baseline.

Phase 8 — Full Ablation Matrix

Once all models exist:

1. **Sparsity sweep:** {1%, 5%, 10%, 25%, 100%} × {all models}
2. **Noise sweep:** Gaussian noise σ ∈ {0%, 2%, 5%, 10%} added to observed sensor values × {all models}
3. For each (model, sparsity, noise) combination, log: relative L_2, relative H^1, PDE residual norm, enstrophy trajectory $\Omega(t)$ vs. ground truth, and radial energy spectrum $E(k)$ vs. ground truth.
4. This is likely the most compute-heavy phase — **budget Colab GPU-hours carefully**; consider whether the full cross-product is affordable or whether a reduced grid (e.g. sparsity sweep at fixed noise=0%, then noise sweep at fixed sparsity=5%) is more realistic given time constraints.

Phase 9 — Figures & Diagnostics for the Paper

1. **1D radial energy spectrum plot:** $E(k)$ vs. k (log-log), ground truth + all models overlaid, with the Kolmogorov k^{-3} reference slope drawn in. This is the paper's headline figure.
2. **Enstrophy dissipation plot:** $\Omega(t)$ vs. time, ground truth + all models.
3. **Gradient norm pathology plot:** $\|\nabla_\theta \mathcal{L}_{data}\|_2$ vs. $\|\nabla_\theta \mathcal{L}_{pde}\|_2$ over training epochs, Static PIFNO vs. GB-PIFNO (from Phase 5 logs).
4. **Sparsity/noise degradation curves:** relative L_2/H^1 vs. sparsity level (and separately vs. noise level), all models overlaid — this is what demonstrates the "data-only models collapse, GB-PIFNO stays robust" claim.
5. **Qualitative comparison panels:** side-by-side vorticity field snapshots (ground truth vs. each model's prediction) at a challenging sparsity level, to visually show over-smoothing vs. preserved detail.

Phase 10 — Paper Writing

1. Related work section — cite PINO, PDEBench, CAPE-FNO, U-NO/F-FNO, PINTO, PDEInvBench, Wang et al. 2021 (gradient norm annealing — the theoretical precedent

for GB-PIFNO's approach), DeepONet, CNO (per the literature review already compiled).
2. Method section — formalize the math already defined in §1 above.
3. Results section — populate with Phase 8/9 outputs.
4. Ablation/discussion — sparsity and noise robustness arguments, framed around Phase 8 results.

---

6. Known Risks / Things to Watch

- **VRAM during Phase 5:** gradient-norm computation for balancing requires extra backward passes per step. Monitor memory; drop batch size before dropping model width if OOM occurs (preserves architectural comparability with other models).
- **Static PIFNO may genuinely fail to train** (output collapse) — this is expected per the proposal's own motivation, not something to silently "fix" by weakening the test; document it as evidence.
- **Forcing term correctness (Phase 4)** is the single most likely silent-bug source — if the residual doesn't include the exact forcing function from §3, all downstream PDE-loss-based results (Static PIFNO, GB-PIFNO) will be built on a biased signal without any obvious symptom.
- **Tier 3 models (PINO, PINTO, CNO) are cited, not implemented** — confirm the whole team agrees on this scope boundary before writing the related-work section, so no one duplicates PINO thinking it's a separate required baseline.
- **Compute budget for Phase 8's full ablation grid** — plan GPU-hours before committing to the full cross-product of sparsity × noise × 6 models.

---

7. Suggested Team Division (adjust to actual team size/skills)

Workstream	Phases	Depends on
A — Physics core	4, 5	Completed infra (§4)
B — Additional baselines	6, 7	Completed infra (§4), parallel to A
C — Evaluation & figures	8, 9	A + B complete
D — Writing	10 (ongoing)	Can start related-work/method sections immediately, in parallel with everything else

---

8. Codebase Reference (files that exist today)

File	Contents
`fno.py`	`SpectralConv2d`, `FNO2d`
`unet.py`	`UNet2d`
`eval_metrics.py`	`relative_l2_error`, `fourier_gradients`, `h1_relative_error`, `radial_energy_spectrum`, `enstrophy`
`sparsity.py`	`generate_pixel_mask`, `apply_sparsity`
`train.py`	`train_data_only`
(to be created)	`pde_residual.py` (Phase 4), `train_static_pifno.py` (Phase 4), `train_gb_pifno.py` (Phase 5), `deeponet.py` (Phase 6), ablation runner + plotting scripts (Phase 8/9)